计算机学院 并行程序设计实验报告

Lab2 SIMD 编程：ANN 近似最近邻搜索

AVX2 / NEON 向量化、量化检索与 FastScan 扩展

柯云超 2413575 计算机科学与技术 2026 年 4 月 18 日

摘要

本文围绕 ANN（Approximate Nearest Neighbor）选题完成 SIMD 编程实验，在 ARM NEON 与 x86 AVX2 平台上实现并比较了四类检索方法：串行 Flat、Flat-SIMD、SQ-SIMD、PQ-SIMD，并在 Windows i9-13900H 上进一步实现了基于 4-bit PQ 的 FastScan 扩展。针对 SIMD 编程实验的基础要求，本文不仅给出向量化算法设计与实现，还结合已有实验数据讨论了内存对齐、软件预取、问题规模变化、自动向量化与手写 SIMD、以及 VTune / 汇编层面的性能原因。实验使用 DEEP100K 数据集（$N = 100000$，$d = 96$，IP 距离），统一以前 2000 个查询、$k = 10$ 进行评测。

结果表明：在 Windows i9-13900H 上，Flat-AVX2 将串行基线从 4571.62 μs 降至 1756.11 μs；SQ-AVX2 在 $p = 100$ 时以无损 recall 达到 427.01 μs，取得 10.71× 加速；PQ-AVX2 在 $p = 1000$ 时达到 761.99 μs、Recall@10 为 0.98330。手写 AVX2 相对标量禁向量化版本达到 8.97× 加速，相对自动向量化版本仍有 6.98× 优势。VTune 表明 Flat-AVX2 已进入明显的带宽受限区间，而 SQ/PQ 更偏向计算与 cache 平衡。进一步地，FastScan 在 Windows P-core 绑核实验中于 $p = 40 \sim 1000$ 区间保持约 527–542 μs 的近似常数时延，在 $p = 5000$ 时以 0.99875 的 Recall@10 将时延降至 674.16 μs，比标准 PQ 快 4.51×，比 Flat-AVX2 快 2.60×。这些结果说明，ANN 场景中的 SIMD 优化应与量化压缩、数据布局和微体系结构特性联合设计，而不仅仅依赖单一的向量指令替换。

关键词： ANN，SIMD，AVX2，NEON，标量量化，乘积量化，FastScan，自动向量化

1 问题描述与实验目标

本次 SIMD 编程实验选择 ANN 近似最近邻搜索任务。给定基向量集合

$$\mathcal{B} = \{x_0, x_1, \ldots, x_{N-1}\}, \quad x_i \in \mathbb{R}^{96},$$

以及查询向量 q，实验目标是在 IP（Inner Product）度量下寻找 top-k 最相近的基向量。串行 Flat 的距离定义为

$$d_{\mathrm{IP}}(x, q) = 1 - \sum_{j=0}^{d-1} x_j q_j.$$

直接全扫描的理论复杂度为 $O(Nd)$。当 $N = 100000$、$d = 96$ 时，单次查询需要完成大量浮点乘加和连续内存访问，既适合使用 SIMD 指令做数据级并行，也适合引入量化与两阶段检索降低代价。因此，本文将大问题拆成以下子问题：

1. 对最基础的 Flat 全扫描进行手写 SIMD 并行化；
2. 基于标量量化（SQ）和乘积量化（PQ）构造 “粗排 + 精排” 两阶段检索；
3. 对比不同编程策略，包括内存对齐、软件预取和自动向量化；
4. 在 x86 与 ARM 平台上比较 SIMD 并行化效果；
5. 使用 VTune、编译器向量化报告和汇编输出来分析性能差异的根本原因；
6. 在基础方法之外，引入 4-bit PQ FastScan 作为进一步的结构化优化。

Git 项目链接为：`https://github.com/kyc001/parallel/tree/master/ann`。

2 实验平台、数据集与测量方法

2.1 平台与软件环境

本文主要使用三个平台的数据：Windows i9-13900H（AVX2，主开发与 VTune 平台）、鲲鹏 920 服务器（NEON，正式 ARM 提交平台）以及历史平板 Termux 数据（NEON，保留用于对齐 / 预取 / 规模实验与横向对照）。实验环境如表 2.1 所示。

平台	ISA / SIMD	编译器与工具	主要用途
Windows 11, i9-13900H	x86_64, AVX2 + FMA	MSYS2 MinGW-w64 g++ 14.2.0, VTune 2025	AVX2 实现、P-core 绑核速度测试、VTune 与汇编分析
鲲鹏 920 服务器	AArch64, NEON	课程服务器环境，`bash test.sh 1 1` 提交	ARM 正式实验数据采集
Android 平板 Termux 历史环境	AArch64, NEON asimd/asimddp	Termux + PRoot Ubuntu	对齐、prefetch、规模实验与历史交叉验证

表 2.1 实验平台与用途

2.2 数据集与评测指标

实验使用 DEEP100K 数据集：100K 条 96 维 float32 基向量、10K 条查询向量，以及对应 top-100 ground truth。所有正式速度实验统一取前 2000 个查询，$k = 10$。性能评价采用平均单查询时延（μs）与 Recall@10。Recall@10 定义为返回的 top-10 中命中 ground truth top-10 的比例。

2.3 测量方式

Windows 平台的主要 AVX2 实验均在单线程下运行；FastScan 结果额外采用 PowerShell 设置处理器亲和性到 P-core 进行校正。鲲鹏结果使用课程要求的 `test.sh` 提交方式得到。对于自动向量化对比实验，本文单独构造了 96 维点积微基准并重复调用 200 万次，以纳秒级平均耗时评估标量、自动向量化与手写 SIMD 三种版本。

3 算法设计与实现

3.1 串行 Flat 与 Flat-SIMD

串行 Flat 的核心是对每个基向量执行一次 96 维点积，复杂度为 $O(Nd)$。其优点是完全精确、实现直接，但访存量大且单次查询需要遍历全部 100K 向量，容易进入带宽受限状态。

Flat-SIMD 的思路是在点积阶段直接使用向量寄存器并行处理多个 float。本文在 AVX2 路径中一次处理 8 个 float，并采用 4 个独立累加器减少流水线依赖；在 NEON 路径中一次处理 4 个 float，同样使用 4 路展开。以 AVX2 为例，核心结构可概括为：

Listing 1 Flat-AVX2 的四累加器点积结构（示意）

```
for (; i + 32 <= d; i += 32) {
    sum0 = _mm256_fmadd_ps(load(x + i),      load(y + i),      sum0);
    sum1 = _mm256_fmadd_ps(load(x + i + 8),  load(y + i + 8),  sum1);
    sum2 = _mm256_fmadd_ps(load(x + i + 16), load(y + i + 16), sum2);
    sum3 = _mm256_fmadd_ps(load(x + i + 24), load(y + i + 24), sum3);
}
```

这种写法的理论复杂度不变，仍是 $O(Nd)$，但常数显著降低。

3.2 SQ-SIMD：标量量化 + 两阶段检索

SQ（Scalar Quantization）对每一维分别统计最小值和最大值，并将 float32 量化到 uint8：

$$\hat{x}_j = \mathrm{clip}\left(\mathrm{round}\left((x_j - v_{\min,j}) \cdot \frac{255}{v_{\max,j} - v_{\min,j}}\right)\right).$$

检索阶段分两步完成：

1. 粗排：将查询同样量化成 uint8，使用 SIMD 计算 96 维 uint8 L2 平方距离，保留 top-p 候选；
2. 精排：仅对 top-p 候选重新计算精确 float32 IP 距离，返回 top-k。

因此 SQ 的复杂度可写为

$$T_{\mathrm{SQ}} = O(Nd_{\mathrm{u8}}) + O(pd_{\mathrm{f32}}),$$

其中第一项由粗排主导，第二项由 rerank 主导。由于本实验数据集中 SQ 的量化误差很小，其粗排顺序与原始 IP 顺序高度一致，因此小 p 下即可保持极高 recall。

3.3 PQ-SIMD：乘积量化 + ADC 检索

PQ（Product Quantization）将 96 维向量分为 $M = 8$ 个子段，每段维度 $d_{\mathrm{sub}} = 12$。对每个子段分别做 k-means 聚类（本实验取 $k = 256$），编码时每个子段只保存 1 个字节的聚类中心编号，故每个向量仅需 8 字节。查询时先构建查找表（LUT），再使用 ADC（Asymmetric Distance Computation）估计距离：

$$\tilde{d}(x, q) = 1 - \sum_{m=1}^{M} \mathrm{LUT}_m[\mathrm{code}_m(x)].$$

PQ 的粗排复杂度约为 $O(NM)$，比 SQ 的 $O(Nd)$ 更低；但由于量化更激进，小 p 时 recall 会明显下降。PQ 的优势在于：粗排极快、内存压缩率高、适合在更大规模数据上扩展。

3.4 FastScan 扩展：4-bit PQ 与查表打包

为了进一步降低 PQ 的粗排常数项，本文在 Windows AVX2 上实现了 FastScan 扩展，并额外补写了可在 AArch64 NEON 上编译的版本。该方法采用 $M = 16$、$K_s = 16$ 的 4-bit PQ，将每两个子段的 code 打包进一个字节，并把 LUT 量化到 0–15 的小整数。粗排阶段不再使用 float LUT，而是使用字节查表与饱和累加：

- AVX2 路径使用 `_mm256_shuffle_epi8`；
- NEON 路径使用 `vqtbl1q_u8`。

这样做的直接收益有两点：其一，LUT 从标准 PQ 的 $M \times 256 \times 4$ Byte 缩小为 $M \times 16$ Byte；其二，批量扫描时可显著提升 cache 友好性和查表吞吐。FastScan 的总复杂度仍可写成

$$T_{\mathrm{FS}} = O(NM) + O(pd),$$

但其中粗排部分的常数显著小于标准 PQ。

方法	每向量存储	粗排主要代价	备注
Flat / Flat-SIMD	384 B	$O(Nd)$ float32 点积	精确检索
SQ-SIMD	96 B	$O(Nd)$ uint8 L2	4× 压缩，recall 很稳
PQ-SIMD ($M = 8$)	8 B	$O(NM)$ ADC 查表	48× 压缩
FastScan ($M = 16$, 4-bit)	8 B	$O(NM)$ 小 LUT 查表 + 打包累加	LUT 更小，粗排常数更低

表 3.2 几种方法的复杂度与存储开销比较

4 实验结果与分析

4.1 核心方法对比：Windows AVX2 与鲲鹏 NEON

表 4.3 汇总了当前最具有代表性的两组核心对比结果：Windows i9-13900H AVX2 与鲲鹏 920 NEON。为了保证 recall 与延迟的平衡，SQ 取 $p = 100$，PQ 取 $p = 1000$。

方法	Recall@10	i9 AVX2 (μs)	鲲鹏 NEON (μs)	i9 相对基线	鲲鹏相对基线
Baseline Flat	0.99995	4571.62	16120.74	1.00×	1.00×
Flat-SIMD	0.99995	1756.11	5821.16	2.60×	2.77×
SQ-SIMD ($p = 100$)	0.99995	427.01	2422.68	10.71×	6.65×
PQ-SIMD ($p = 1000$)	0.98330 / 0.98355	761.99	1984.35	6.00×	8.12×

表 4.3 核心方法在 i9 AVX2 与鲲鹏 NEON 上的对比

可以看到，Flat-SIMD 的提升主要来自向量化后的点积吞吐提高，但由于仍需全量扫描 100K 向量，其收益最终受制于内存带宽。相比之下，SQ 与 PQ 通过两阶段检索降低了精确距离计算次数，因此时延下降更明显。SQ 在本数据集上的一大特点是 recall 非常稳定；PQ 的小 p 区间 recall 会下降，但当 p 提升到 1000 以上后可恢复到较高准确率。

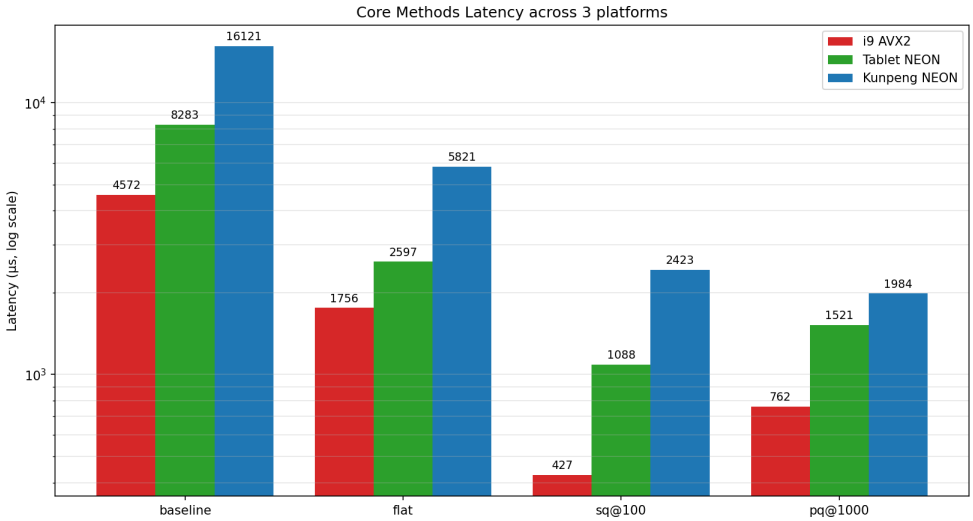

图 4.1 三平台核心方法对比图（已有实验结果整理）

4.2 SQ / PQ 的时延-召回率权衡

Windows i9 AVX2 的完整调优结果表明，SQ 与 PQ 呈现出两种不同的 tradeoff 曲线。SQ 在整个 p 扫描区间里 recall 基本不变，而延迟近似随 p 线性上升；PQ 则体现出更典型的“小 p 更快但 recall 下降”特征。图 4.2 给出 i9 上的对比图。

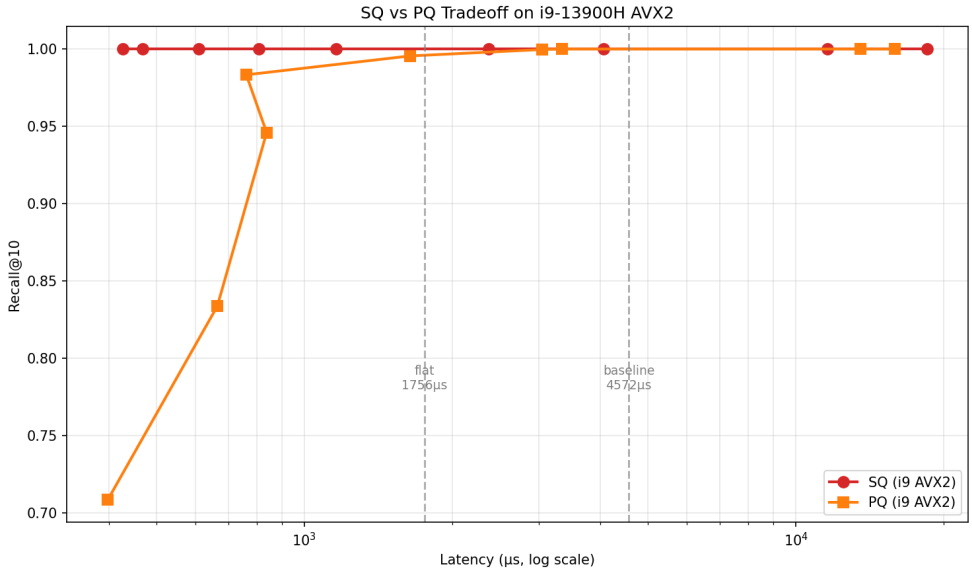

图 4.2 i9-13900H 上 SQ-AVX2 与 PQ-AVX2 的时延-召回率权衡

从数值上看：

- SQ-AVX2 在 $p = 100$ 时仅 427.01 μs，Recall@10 仍为 0.99995；
- PQ-AVX2 在 $p = 100$ 时更快（398.53 μs），但 Recall@10 仅为 0.70860；
- PQ-AVX2 在 $p = 1000$ 时达到 761.99 μs、Recall@10 为 0.98330，是更均衡的 PQ 配置；
- 当 p 非常大时，两阶段方法的 rerank 成本逐渐吞噬收益，速度优势减弱。

4.3 对齐、软件预取与问题规模

为了覆盖实验指导书中要求的“基本算法 / 编程策略对性能的影响”，本文保留并整理了 ARM 历史环境中 Flat-SIMD 的 E1/E2/E3 实验结果。

对齐与非对齐。 E1 使用 64B 对齐、默认 `new[]` 和 1 字节偏移三种方式分配 Flat-SIMD 的基向量内存。结果表明，AArch64 NEON 对非对齐访问具有硬件容忍性，但强行制造 1B 偏移会带来约 6.45% 的性能损失，如表 4.4 所示。这说明“能跑”不等于“无代价”，对齐仍然影响 cache line split 和访存效率。

软件预取。 E2 在 Flat-SIMD 流式扫描上尝试多种预取距离。结果显示该 workload 上显式软件预取并未带来收益，反而在所有 tested distance 下均略慢于无预取基线。这表明当前访问模式已经足够顺序，硬件预取器能够较好地覆盖需求，额外预取指令只会增加额外负担。

实验项	中位总时延（2000 queries, μs）	相对基线
E1: 64B 对齐	6238505	1.0000×
E1: 默认 `new[]`	6143142	1.0155×
E1: 1B 偏移非对齐	6668860	0.9355×
E2: 无预取	6219560	1.0000×
E2: 预取距离 4	6220804	0.9998×
E2: 预取距离 8	6752636	0.9211×
E2: 预取距离 16	6485157	0.9591×
E2: 预取距离 32	6477509	0.9602×

表 4.4 ARM 历史环境中的对齐 / 预取实验结果

问题规模。 E3 表明 Serial Flat 与 Flat-SIMD 的时延近似随 N 线性增长，符合全扫描的 $O(Nd)$ 复杂度；而 SQ / PQ 的优势只有在基向量规模足够大时才更明显。需要注意的是，E3 在 $N < 100000$ 时复用了完整 100K 的 ground truth，因此 recall 数值天然偏低，不能与完整数据规模下的 recall 直接比较；这部分结果主要用于观察规模趋势而非做准确率结论。

4.4 手写 SIMD 与自动向量化对比

根据实验指导书的进阶要求，本文额外构造了单独的 96 维点积微基准，对比三种实现：禁用自动向量化的标量版本、允许编译器自动向量化的版本，以及手写 AVX2 版本。表 4.5 给出结果。

版本	单次调用耗时（ns）	相对标量加速比
标量禁向量化	27.9491	1.00×
自动向量化	21.7258	1.29×
手写 AVX2	3.11465	8.97×

表 4.5 Windows i9-13900H 上手写 SIMD 与自动向量化对比

编译器向量化报告显示，自动向量化版本的确完成了 256-bit 向量化，日志中出现了 “loop vectorized using 32 byte vectors”。然而，实测加速仅为 1.29×。与之相比，手写 AVX2 版本相对自动向量化仍快 6.98×。其根本原因不在于 “有没有向量化”，而在于向量化后的代码形态是否充分利用微体系结构：手写版本通过多累加器、显式 FMA 和更规整的依赖链组织，显著提升了流水线利用率，而自动向量化版本虽然产生了 AVX2 指令，却未达到同样的调度质量。

4.5 VTune 与汇编层面的性能剖析

Windows i9-13900H 上的 VTune 结果进一步解释了三类方法的瓶颈差异。表 4.6 汇总最关键的 Top-Down 指标。

版本	CPI	Retiring	Front-End	Back-End	Memory Bound	关键特征
Flat-AVX2（P-core）	0.904	19.5%	1.5%	71.4%	56.3%	256-bit FP uOps 33.8%
SQ-AVX2 ($p = 100$)	0.307	47.5%	5.9%	23.0%	8.7%	256-bit Int SIMD 28.6%
PQ-AVX2 ($p = 1000$)	0.270	68.5%	8.0%	16.7%	0.6%	3+ ports utilized 85.9%

表 4.6 Windows i9-13900H 上的 VTune 微体系结构结果

这组数据给出三个重要结论：

1. Flat-AVX2 已经显著带宽受限。P-core 绑核后平均频率升至 5.086 GHz，但 Memory Bandwidth 仍占到 91.9% 时钟，DRAM 峰值带宽约 42.1 GB/s。这意味着继续优化浮点乘加本身的收益有限，系统瓶颈已经主要转移到数据搬运。
2. SQ-AVX2 是三者中最均衡的配置。它充分使用 256-bit 整数 SIMD 粗排，又把 rerank 控制在较小的 p，因此能在几乎无损 recall 的前提下取得最优的单线程速度。
3. PQ-AVX2 查询阶段本身很高效，Retiring 比例最高，但端到端程序时间还包含建索引的 k-means 训练开销。若仅看查询阶段，ADC 查表的吞吐是非常强的。

此外，编译器输出与 VTune 还暴露出一个很有意思的点：Flat 和 SQ 中都能观察到一定程度的 Machine Clears，推测与 `priority_queue` 维护 top-k 时的分支和 store-load 依赖有关。这说明在 ANN 这种非纯数值内核里，SIMD 之外的数据结构与控制流也会成为实际瓶颈。

4.6 FastScan 扩展实验：Windows 已完成，Kunpeng 待补测

FastScan 是本文在基础要求之外新增的一项结构化优化。Windows i9-13900H P-core 绑核实验结果如表 4.7 所示。

p	Recall@10	FastScan (μs)	相对标准 PQ 加速
40	0.51400	536.447	–
100	0.70075	529.995	–
500	0.93205	526.872	1.59×
1000	0.97535	541.689	1.41×
2000	0.99265	571.850	2.87×
5000	0.99875	674.161	4.51×

表 4.7 Windows i9-13900H 上 PQ-FastScan 的正式结果（P-core 绑核）

FastScan 曲线最有价值的地方在于：从 $p = 40$ 到 $p = 1000$，Recall@10 从 0.514 提升到 0.975，而时延仍基本稳定在 527–542 μs。也就是说，其粗排主要成本与 p 无关，p 的增长只在较大范围内逐渐反映为 rerank 成本上升。与标准 PQ 对比，FastScan 在小 p 区间由于固定开销较高不一定占优，但一旦 p 进入中大区间就迅速反超；在 $p = 5000$ 时，FastScan 以 0.99875 的 Recall@10 将标准 PQ 的 3042.08 μs 压缩到 674.16 μs。相较 Flat-AVX2 的 1756.11 μs，FastScan 在近乎满 recall 下仍可取得 2.60× 加速。

目前本文已在本地补写了 NEON 版本的 FastScan 头文件与可直接配合 `test.sh` 提交的包装脚本，鲲鹏平台的正式结果将在后续补测后并入表格。

5 综合讨论

将上述结果合并起来，可以得到四点更一般的认识：

1. **Flat-SIMD 的收益会被带宽上限限制。**当算法仍需全量扫描原始 float32 数据时，SIMD 虽能降低计算常数，却无法消除对主存带宽的依赖。
2. **量化方法的收益不仅来自 SIMD，更来自算法结构变化。** SQ 与 PQ 之所以比 Flat 快很多，关键不只是 SIMD 化了单次距离计算，还在于把大部分候选提前在压缩空间中筛掉了。
3. **SQ 与 PQ 对应不同应用目标。** SQ 在本数据集上几乎无损 recall，适合高准确率优先的场景；PQ 更像可调的 tradeoff 旋钮，适合在准确率与速度之间动态取舍。
4. **手写 SIMD 与数据布局联合优化仍然必要。**自动向量化只能给出基础收益，而 FastScan 这类需要重构编码布局和 LUT 结构的优化，必须由程序员显式设计。

6 结论与后续工作

本文完成了 ANN 选题在 SIMD 编程实验中的主要基础要求，并在多个方向上做了进阶扩展。基础部分方面，本文在 ARM NEON 与 x86 AVX2 上实现并测量了 Flat-SIMD、SQ-SIMD 与 PQ-SIMD，整理了完整的时延-召回率曲线；同时使用对齐 / 预取 / 规模实验、VTune 与汇编分析解释了性能差异。进阶部分方面，本文完成了手写 SIMD 与自动向量化对比实验、跨平台对比分析，并新增实现了 FastScan 扩展。当前结果表明，SQ-AVX2 ($p = 100$) 是整体上最稳健的配置，而 FastScan 在中高 recall 区间展示出最强的延迟优势。

下一步工作有两项：第一，在鲲鹏 920 上提交 FastScan 封装脚本，补齐 ARM FastScan 的正式结果；第二，视课程时间与平台权限情况，继续尝试更细粒度的 top-k 维护优化，以减少 heap 分支与 Machine Clears 对端到端时延的影响。

A AI 辅助开发说明

本次实验使用生成式 AI 辅助了以下工作：梳理 SIMD 实验要求、讨论 SQ / PQ / FastScan 的实现思路、补写 Windows AVX2 与 AArch64 NEON 的封装代码、整理自动向量化与汇编分析的实验入口、以及协助撰写实验报告初稿。完整对话记录将作为附录材料单独提交。需要强调的是，具

体算法选择、参数设计、代码落地、结果验证与性能分析均由本人结合本地与服务器实验结果独立完成。

参考文献

[1] Fabien André, Balázs Kégl, Mickael Rouvier, and Herve Jegou. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proceedings of the VLDB Endowment*, 9(4):288–299, 2015.

[2] Artem Babenko and Victor Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2055–2063, 2016.

[3] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[4] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.